

Organizzazione Fisica dei Dati (Parte II)

Angelo Montanari

Dipartimento di Matematica e Informatica
Università di Udine

Strutture ad indice per file

Gli **indici** (index) sono strutture di accesso ai dati utilizzate per rendere più veloce il recupero di record in presenza di determinate condizioni di ricerca.

Alcuni tipi di indici (secondary access path) non influenzano la disposizione fisica dei record sul disco, ma definiscono dei cammini di ricerca alternativi per localizzare efficientemente i record sulla base di opportuni campi indice (indexing field).

I più importanti tipi di indice sono basati su file ordinati (indici di singolo livello) e su strutture dati ad albero (indici multilivello statici, B -alberi, B^+ -alberi).

Gli indici possono essere costruiti sulla base di strutture dati di tipo hashing o di altro tipo.

Indici ordinati di singolo livello

L'idea alla base di una struttura ordinata ad indici è simile a quella degli indici per parole che si trovano alla fine dei libri di testo: accanto ad ogni parola vi è la lista dei numeri di pagina dove la parola compare nel testo (alternativamente, bisogna scorrere tutto il testo parola per parola: linear search).

Una struttura ad indici è usualmente definita su un singolo campo di un file (**indexing field**). Ogni indice memorizza per ogni valore del campo una lista di puntatori a blocchi su disco che contengono record con quel valore di campo.

Gli indici sono ordinati in modo da consentire una **ricerca binaria**. Il file indice è molto più piccolo del file di dati: una ricerca binaria sul file indice è molto efficiente.

Diversi tipi di indice (di singolo livello) - 1

Indici primari. Un indice primario è un indice specificato rispetto al campo chiave di un file di record fisicamente ordinato rispetto a tale campo (ordering key field).

Si noti che:

1. tale campo è utilizzato per ordinare fisicamente i record del file sul disco (**ordering** key field);
2. ogni record è identificato univocamente dal valore che assume su tale campo (ordering **key** field).

Diversi tipi di indice (di singolo livello) - 2

Indice di clustering. L'indice di clustering si usa nel caso in cui l'ordering field non sia un key field (più record nel file possono assumere lo stesso valore rispetto all'ordering field).

Osservazione. Poiché un file può essere ordinato rispetto ad al più un **physical ordering field**, esso può avere o un indice primario o un indice clustering, ma non entrambi.

Indice secondario. Un indice secondario può essere definito su ogni campo (non utilizzato per ordinare il file). Un file può avere più indici secondari.

Indice primario - 1

Un **indice primario** è un file ordinato i cui record, di lunghezza prefissata, possiedono due campi:

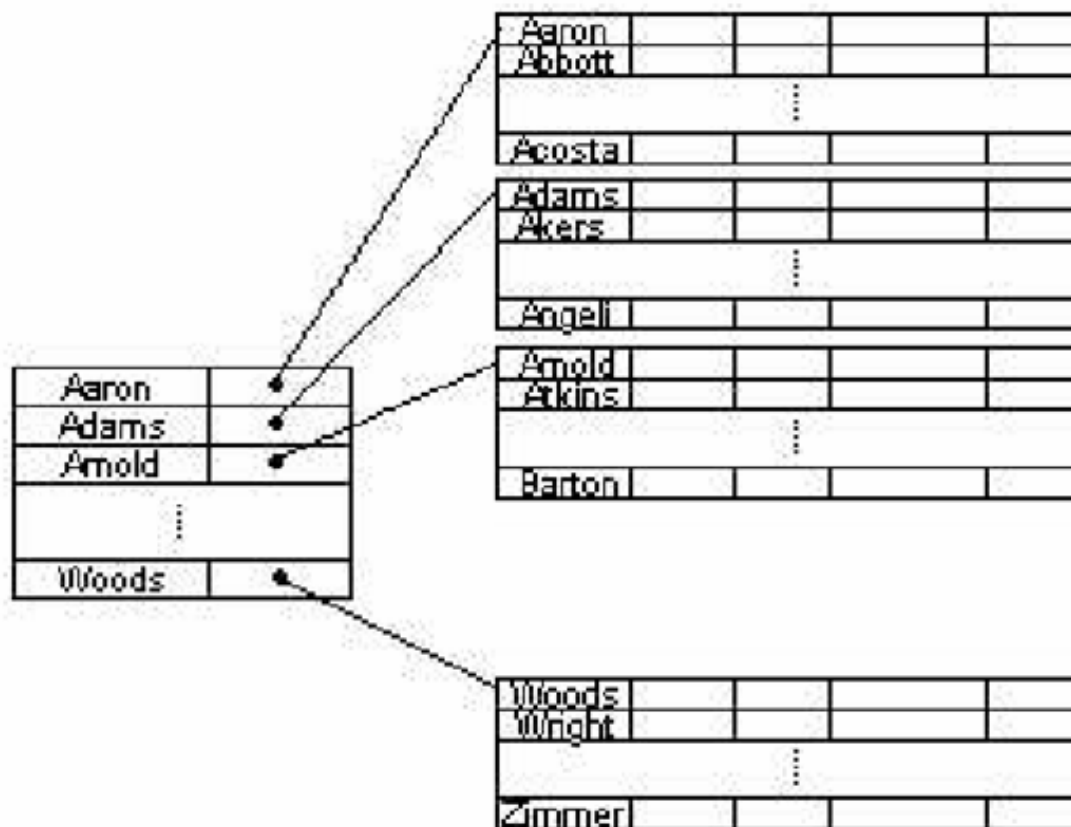
$$(\langle \text{chiave primaria} \rangle, \langle \text{indirizzo blocco} \rangle)$$

Nel file indice esiste una index entry (o index record) i per ogni blocco b del file di dati:

$$\langle k(i), p(i) \rangle$$

dove $k(i)$ è un valore minore (o uguale) a quello di tutte le chiavi dei record in b , ma maggiore del valore di tutte le chiavi del blocco che precede b , e $p(i)$ è l'indirizzo del blocco b .

Indice primario - 3



Indice primario - 4

Il primo record di ogni blocco del file di dati è definito anchor record del blocco (**block anchor**). Un indice primario è un indice **non denso** in quanto include una entry per ogni blocco (un indice è detto denso se contiene una entry per ogni record del file).

Un record con valore della chiave primaria uguale a k si trova nel blocco di indirizzo $p(i)$, con

$$k(i) \leq k < k(i + 1)$$

Per recuperare un record, noto il valore k della sua chiave primaria, occorre effettuare una ricerca binaria sul file indice per cercare l'appropriata index entry i e quindi recuperare il blocco il cui indirizzo è $p(i)$.

Indici di clustering - 1

Se i record di un file sono ordinati fisicamente rispetto ad un campo non chiave (record diversi possono assumere lo stesso valore su tale campo), tale campo è chiamato **clustering field**.

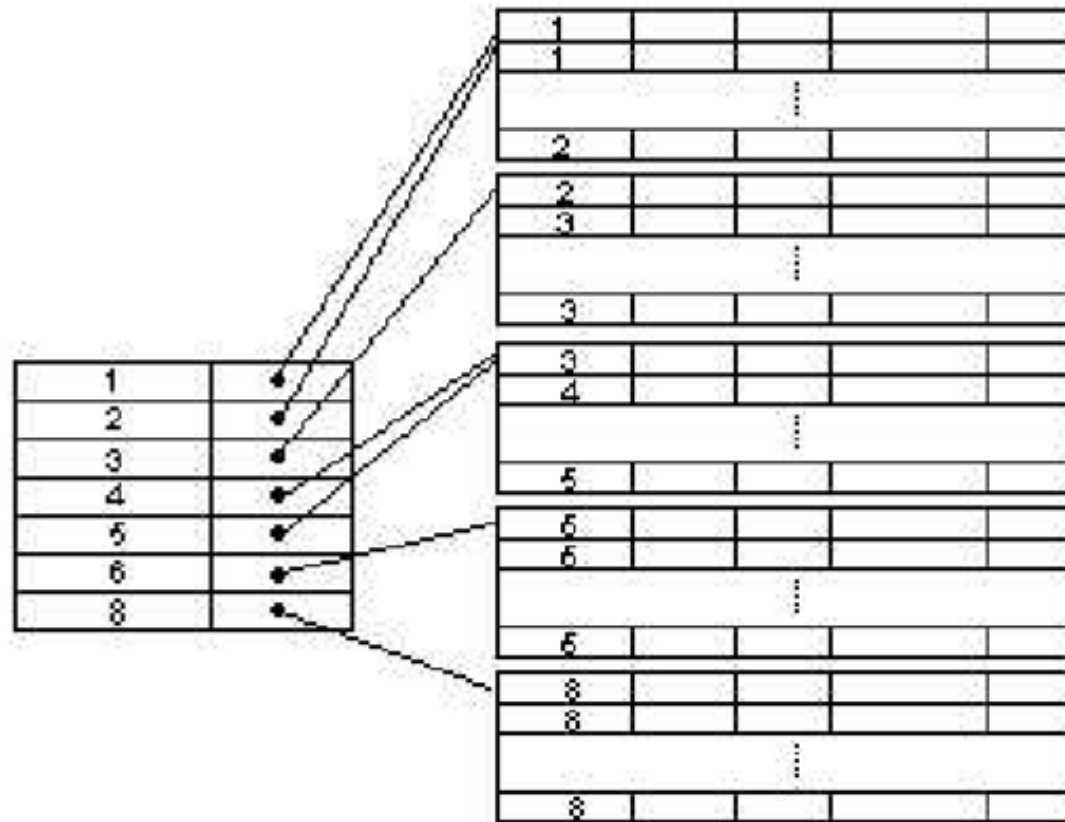
In tal caso, è possibile creare un diverso tipo di indice (**indice di clustering**) per rendere più efficiente il recupero di record che assumono lo stesso valore sul clustering field.

Un clustering index è un file ordinato con due campi:

$$(\langle \text{clustering field} \rangle, \langle \text{indirizzo blocco} \rangle)$$

Esiste una entry nel clustering index per ogni valore distinto del clustering field, contenente il valore ed un puntatore al primo blocco nel file dei dati che ha un record con quel valore per il suo clustering field.

Indici di clustering - 2



Indici secondari - 1

Un indice secondario è (ancora) un file ordinato con due campi:

$(\langle \text{indexing field} \rangle, \langle \text{indirizzo blocco} \rangle)$

A differenza dei due tipi di indici precedenti, è possibile associare più indici secondari (e quindi indexing field) ad uno stesso file.

Consideriamo una struttura con accesso attraverso **indici secondari** su un **campo chiave** (campo che assume un valore diverso su ogni record del file di dati - secondary key).

Vi sarà una index entry per ogni record del file di dati, contenente il valore della chiave secondaria per il record ed un puntatore al blocco in cui il record è memorizzato (puntatore al blocco anziché al record..).

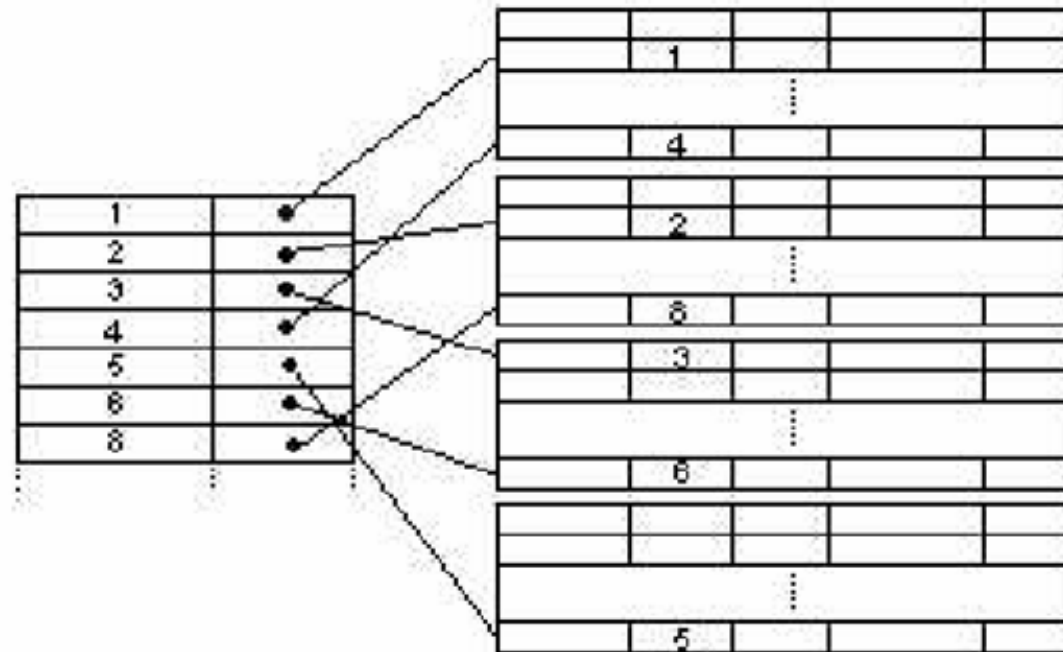
Indici secondari - 2

Un indice secondario su un key field è un indice **denso** (a differenza degli indici primari, gli indici secondari contengono una entry per ogni record, e non per ogni blocco, del file di dati): poiché i record del file di dati non sono fisicamente ordinati rispetto al valore del secondary key field, non è possibile utilizzare i block anchor.

Quando un blocco è trasferito in memoria primaria, bisogna effettuare una ricerca del record desiderato.

Un indice secondario richiede molto più spazio ed un tempo di ricerca maggiore di un indice primario in ragione dell'elevato numero di entry (indice denso). Tuttavia, la riduzione del tempo di ricerca di un record sulla base del valore da esso assunto sull'indexing field è molto significativa (in questi casi, disponendo del solo indice primario, occorre eseguire una scansione lineare).

Indici secondari - 3



Attenzione: si tratta in ogni caso di puntatori a blocco, non a record (è computazionalmente più vantaggioso)

Indici secondari - 6

E' possibile creare un indice secondario anche su un **campo non chiave** (nonkey field) del file. In tal caso, numerosi record possono assumere lo stesso valore sull'indexing field.

Esistono diverse possibili modalità di implementazione di tale indice:

- (1) includere più index entry con lo stesso valore di $k(i)$, una per ogni record (indice denso);
- (2) considerare record di lunghezza variabile per le index entry, con un campo (più volte) ripetuto per il puntatore. Viene mantenuta una lista di puntatori:

$$\langle p(i, 1), \dots, p(i, k) \rangle$$

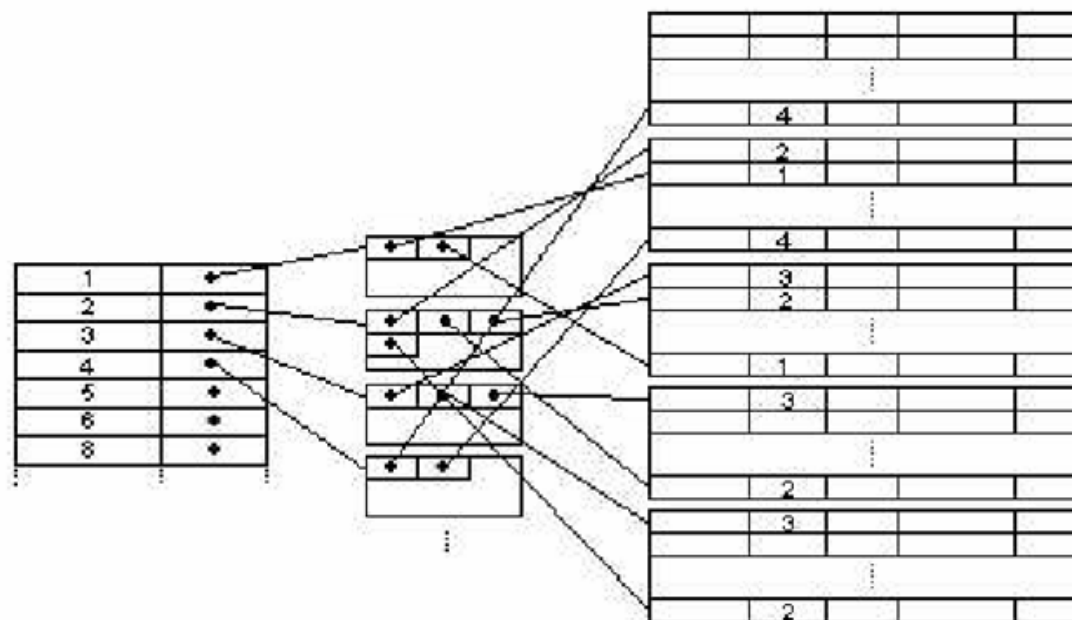
nella index entry per $k(i)$ (un puntatore ad ogni blocco che contiene un record il cui indexing field value è uguale a $k(i)$).

Indici secondari - 7

- (3) Il puntatore $p(i)$ in una index entry $\langle k(i), p(i) \rangle$ punta ad un blocco; ogni puntatore presente nel blocco punta ad (un blocco contenente) un record del file di dati con valore $k(i)$ dell'indexing field.

Se i valori di $k(i)$ sono condivisi da molti record, e, quindi, i corrispondenti puntatori ai (blocchi contenenti i) record non possono essere contenuti in un unico blocco, si utilizza una lista di blocchi (vedi figura riportata nel lucido successivo).

Indici secondari - 8



Indice secondario costruito su un campo non chiave del file (caso 3).

Indici multilivello - 1

Una ricerca binaria richiede approssimativamente $\log_2 b_i$ accessi a blocco per un indice con b_i blocchi, poiché ogni passo dell'algoritmo riduce la parte del file indice da analizzare di un fattore 2.

L'idea di utilizzare **indici multilivello** è nata dalla necessità di ridurre ad ogni passo la parte dell'indice che rimane da analizzare di un fattore pari a bfr_i (che è abitualmente > 2). Il valore bfr_i è detto **fan-out** (fo) dell'indice multilivello.

La ricerca su un indice multilivello richiede approssimativamente $\log_{fo} b_i$ accessi a blocco.

Indici multilivello - 2

Un indice multilivello considera il file indice (primo livello) come un file ordinato con un valore distinto per ogni $k(i)$ e crea un indice primario per il primo livello (**secondo livello**).

Poiché il secondo livello è un indice primario, si possono in realtà usare dei block anchor in modo che il secondo livello abbia una entry per ogni blocco (e non per ogni entry) del primo livello.

Il bfr_i per il secondo livello (e per tutti i livelli successivi) è lo stesso dell'indice del primo livello, poiché tutte le index entry hanno la stessa dimensione (ognuna ha un $\langle field\ value \rangle$ e un $\langle indirizzo\ blocco \rangle$).

Indici multilivello - 3

Se il primo livello ha r_1 entry e $bfr_i = fo$, allora il primo livello ha bisogno di $\lceil r_1/fo \rceil$ blocchi, che è il numero di entry r_2 necessarie al secondo livello dell'indice.

Si può ripetere questo processo per il secondo livello. Il **terzo livello**, che è un indice primario per il secondo livello, ha una entry per ogni blocco di secondo livello.

Il numero di entry al terzo livello è:

$$r_3 = \lceil r_2/fo \rceil$$

Indici multilivello - 4

Osservazione. E' richiesto un secondo livello solo se il primo livello richiede più di un blocco di spazio su disco; è richiesto un terzo livello solo se il secondo richiede più di un blocco, e così via.

Si ripete il procedimento fino a quando tutte le entry di un certo index level t (**top index level**) stanno in un singolo blocco.

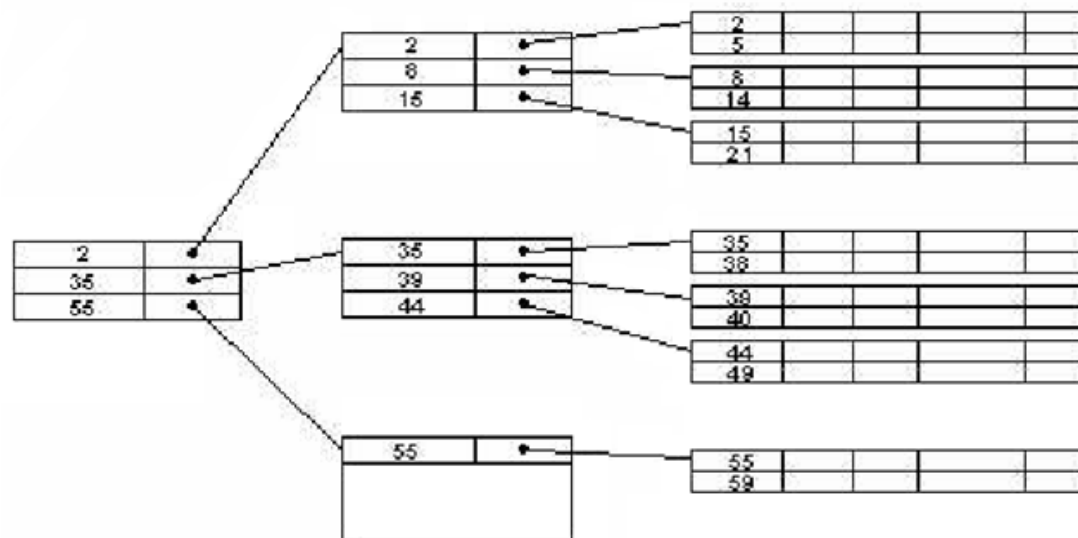
Ogni livello riduce il numero di entry del livello precedente di un fattore f_o . Il top index level t si può calcolare a partire dal vincolo:

$$1 \geq r_1 / f_o^t$$

Un indice multilivello con r_1 entry al primo livello avrà approssimativamente t livelli, dove

$$t = \lceil \log_{f_o} r_1 \rceil$$

Indici multilivello - 5



Alberi di ricerca - 1

Un *albero di ricerca* è un tipo speciale di albero utilizzato per guidare la ricerca di record, dato il valore di uno dei loro campi.

- Un indice multilivello (statico) può essere visto come una variante degli alberi di ricerca.
- Ogni nodo dell'indice multilivello può avere fo puntatori e fo chiavi, con $fo = fan-out$ dell'indice. In ogni nodo, il valore del campo indice consente di passare al nodo successivo finché non si raggiunge il blocco di dati che può contenere il record cercato. Ad ogni livello, si restringe la ricerca ad un sottoalbero radicato nel nodo corrente (ignorando tutti gli altri nodi dell'albero).

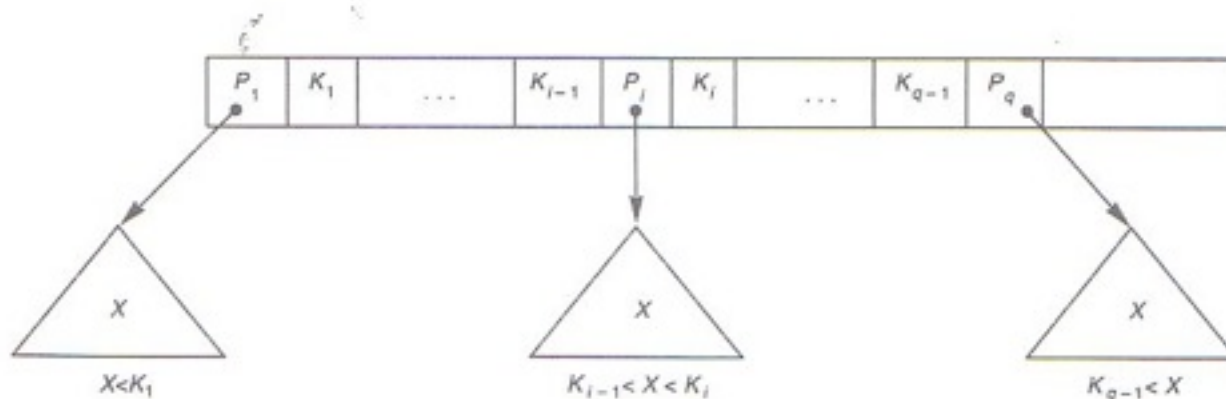
Alberi di ricerca - 2

- Un albero di ricerca di **ordine** p è un albero i cui nodi contengono al più $p-1$ *search value* e p puntatori nel seguente ordine

$$\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle$$

con $q \leq p$.

Ogni P_i è un puntatore ad un nodo figlio (o un puntatore nullo) e ogni K_i è un *search value* appartenente ad un insieme **totalmente ordinato** (assumiamo, per semplicità, che i *search value* K_i siano tutti distinti; non è difficile generalizzare la struttura proposta al caso in cui i *search value* non siano tutti distinti).



Alberi di ricerca - 3

- Ogni albero di ricerca deve soddisfare **due vincoli fondamentali**:

(1) in ogni nodo, $K_1 < K_2 < \dots < K_{q-1}$;

(2) per tutti i valori di X presenti nel sottoalbero puntato da P_i , vale la seguente relazione:

$$- K_{i-1} < X < K_i \quad \text{per } 1 < i < q;$$

$$- X < K_i \quad \text{per } i = 1;$$

$$- K_{i-1} < X \quad \text{per } i = q.$$

Alberi di ricerca - 5

- Un albero di ricerca può essere utilizzato per cercare record memorizzati su disco.
- I valori di ricerca (*search value*) possono essere i valori di uno dei campi del file (*search field*).
- Ad ogni valore di ricerca è associato un puntatore al record avente quel valore (oppure al blocco contenente quel record) nel file di dati.
- L'albero stesso può essere memorizzato su disco, assegnando **ad ogni nodo dell'albero un blocco**. Quando un nuovo record deve essere inserito, l'albero di ricerca deve essere aggiornato includendo il valore del campo di ricerca del nuovo record, col relativo puntatore al record (o al blocco che lo contiene), nell'albero di ricerca.

Alberi di ricerca - 6

- Per *inserire* (risp., *cancellare*) valori di ricerca nell'albero (risp., dall'albero) di ricerca sono necessari specifici algoritmi che garantiscano il rispetto dei due vincoli fondamentali.

In generale, tali algoritmi non garantiscono che un albero di ricerca risulti sempre *bilanciato* (nodi foglia tutti allo stesso livello).

Perché vogliamo alberi di ricerca bilanciati?

un albero di ricerca bilanciato evita che alcuni record richiedano molti più accessi a blocco di altri

- In particolare, la *cancellazione* di record crea nell'albero nodi scarsamente popolati, aumentando la dimensione dello spazio inutilizzato e, conseguentemente, il numero di livelli.
- **Soluzione:** B-alberi e B⁺-alberi.

B-alberi

- Un **B-albero** è un albero di ricerca con alcuni vincoli addizionali i quali garantiscono che l'albero sia bilanciato e lo spazio di memoria che risulta inutilizzato per effetto della cancellazione di record non diventi troppo grande.
- Per soddisfare tali vincoli addizionali, occorrono algoritmi di inserimento e cancellazione di record più complessi, soprattutto nei casi di:
 - inserimento di un record in un nodo pieno;
 - cancellazione di un record da un nodo che diventa pieno per meno del 50% della sua dimensione.

B-alberi: condizioni - 1

- Formalmente, un B-albero di *ordine* p , se usato come struttura di accesso su un campo chiave per la ricerca di record in un file di dati, deve soddisfare le seguenti condizioni:

(1) ogni nodo interno del B-albero ha la forma:

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle;$$

dove $q \leq p$.

P_i è un *tree pointer* (puntatore ad un altro nodo del B-albero), Pr_i è un *data pointer* (puntatore ad un record con valore del campo chiave di ricerca uguale a K_i o al blocco che contiene tale record);

(2) per ogni nodo, si ha che:

$$K_1 < K_2 < \dots < K_{q-1};$$

(3) ogni nodo ha al più p *tree pointer*;

B-alberi: condizioni - 2

(4) per tutti i valori X della chiave di ricerca appartenenti al sottoalbero puntato da P_i , si ha che:

$$- K_{i-1} < X < K_i \quad \text{per } 1 < i < q;$$

$$- X < K_i \quad \text{per } i = 1;$$

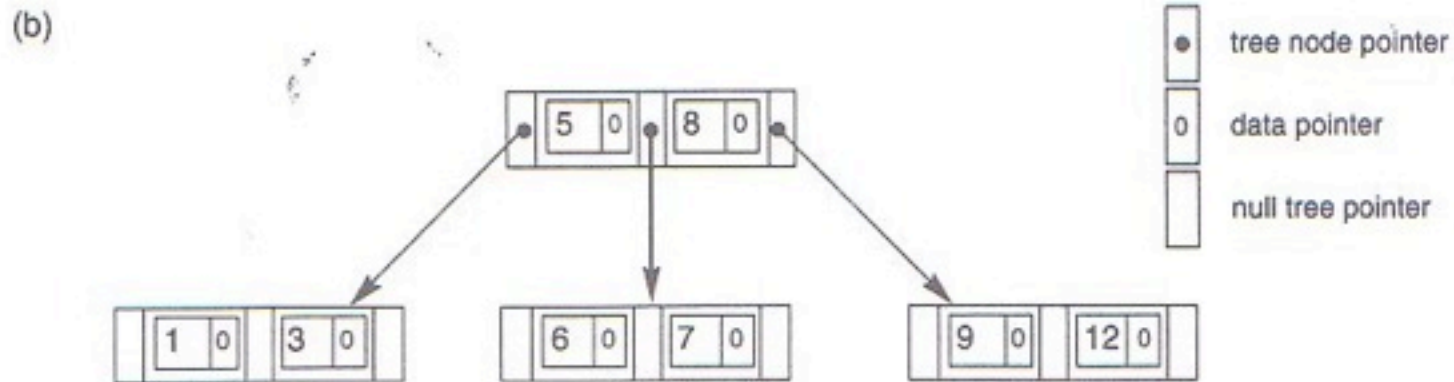
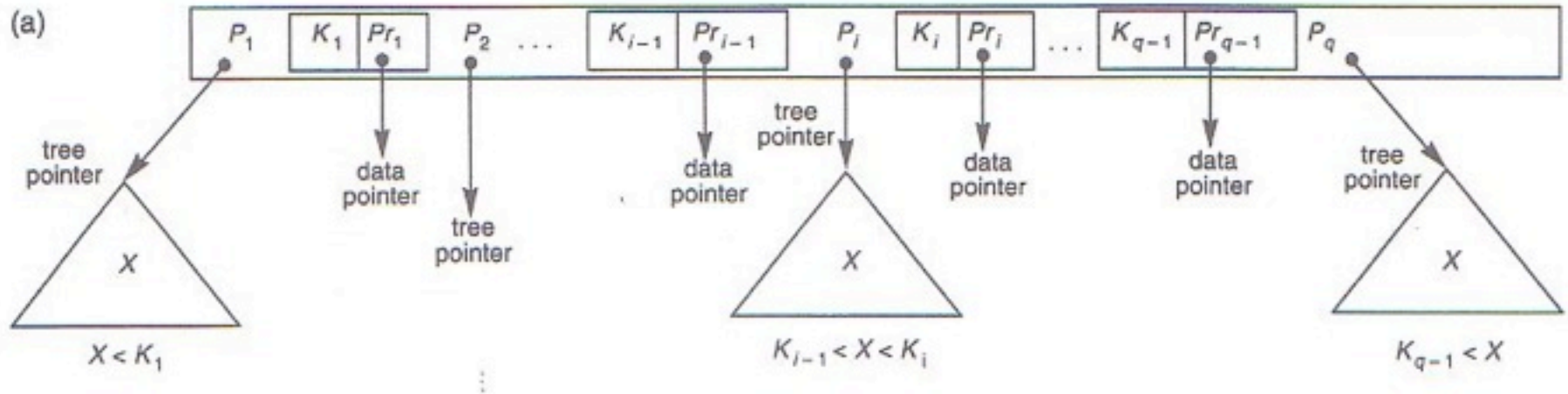
$$- K_{i-1} < X \quad \text{per } i = q;$$

(5) ogni nodo, esclusa la radice, ha almeno $\lceil p/2 \rceil$ tree pointer (la radice ha almeno due *tree pointer*, a meno che non sia l'unico nodo dell'albero);

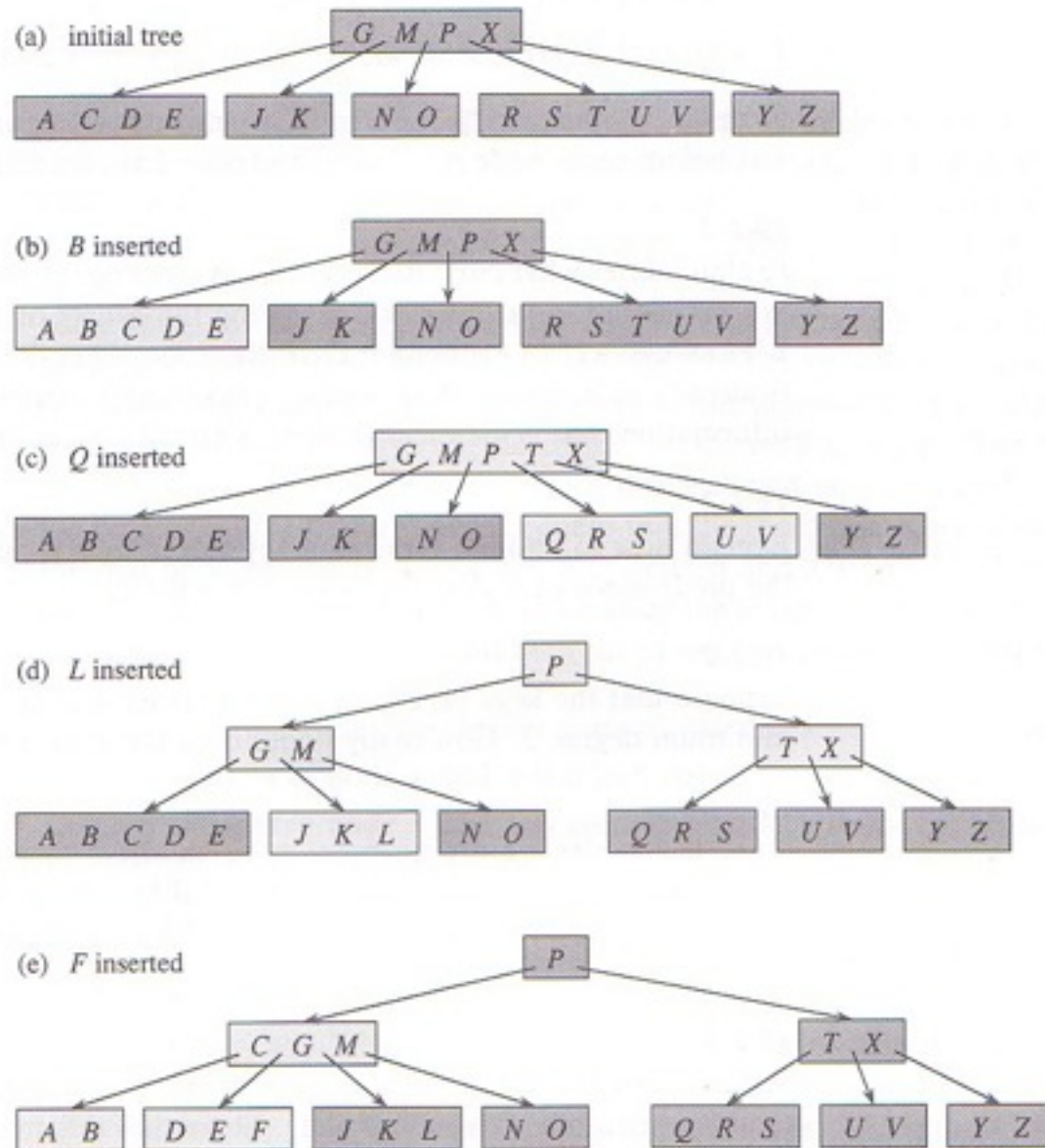
(6) un nodo con q *tree pointer*, con $q \leq p$, ha $q-1$ campi chiave di ricerca (e $q-1$ *data pointer*);

(7) tutti i nodi foglia sono posti allo stesso livello (i nodi foglia hanno la stessa struttura dei nodi interni, ad eccezione del fatto che tutti i loro *tree pointer* P_i sono nulli).

B-alberi: struttura

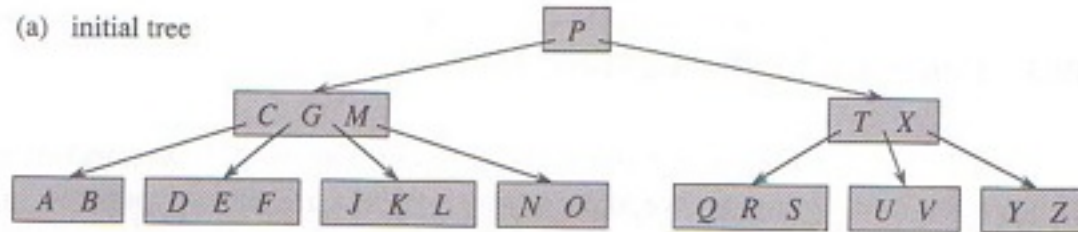


Esempio di inserimento (t=3)

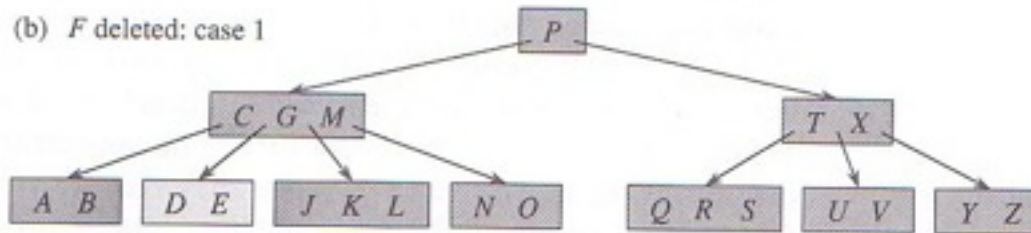


Esempio di cancellazione ($t=3$) - 1

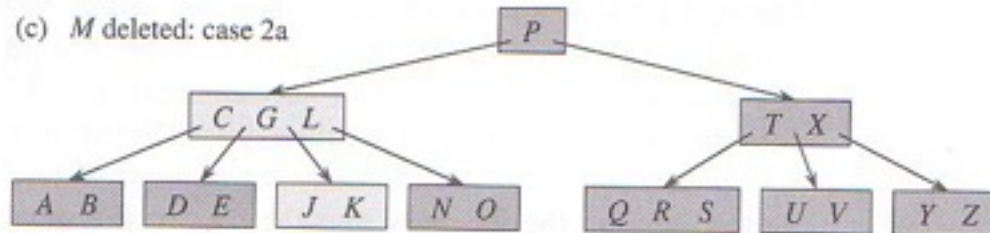
(a) initial tree



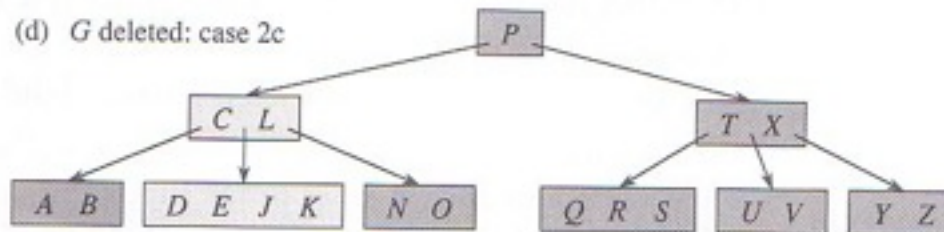
(b) F deleted: case 1



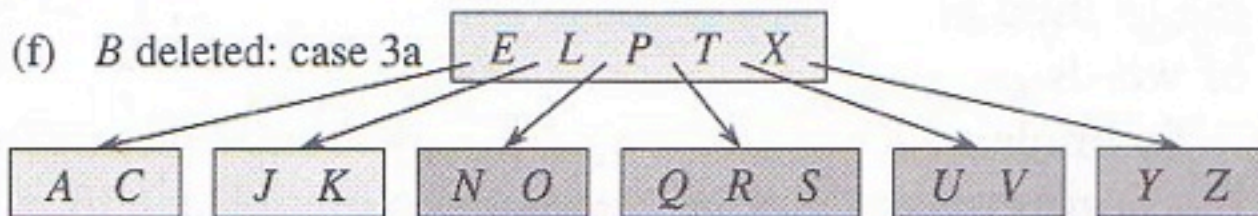
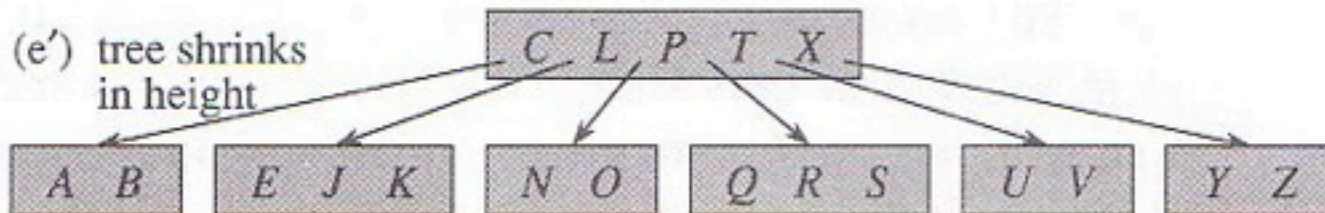
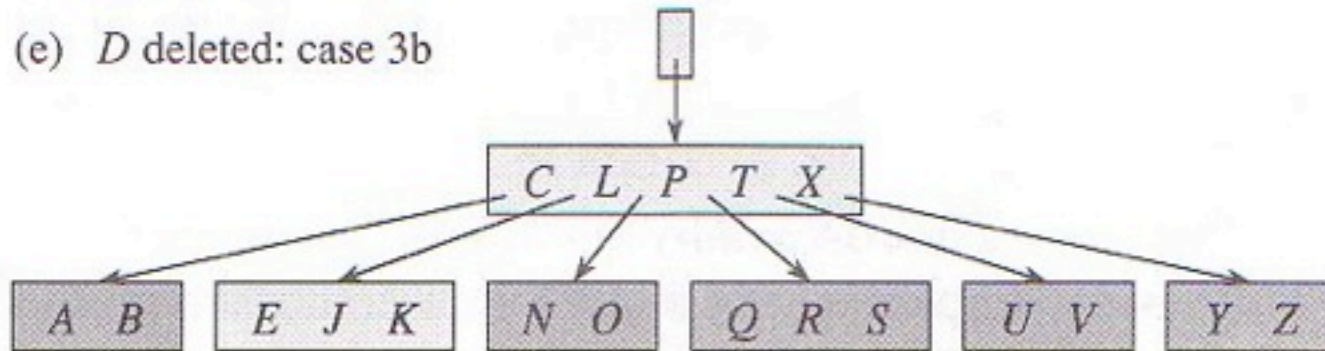
(c) M deleted: case 2a



(d) G deleted: case 2c



Esempio di cancellazione ($t=3$) - 2



Dai B-alberi ai B⁺-alberi

- Un B⁺-albero è un B-albero in cui i data pointer sono memorizzati solo nei nodi foglia dell'albero. La struttura dei nodi foglia differisce, quindi, da quella dei nodi interni.
- Se il campo di ricerca è un campo chiave, i nodi foglia hanno per ogni valore del campo di ricerca una entry ed un puntatore ad un record. Se un campo di ricerca non è un campo chiave, i puntatori indirizzano un blocco che contiene i puntatori ai record del file di dati, rendendo così necessario un passo aggiuntivo per l'accesso ai dati.
- I nodi foglia di un B⁺-albero sono generalmente messi fra loro in relazione (ciò viene sfruttato nel caso di range query). Essi corrispondono al primo livello di un indice. I nodi interni corrispondono agli altri livelli di un indice.
- Alcuni valori del campo di ricerca presenti nei nodi foglia sono ripetuti nei nodi interni per guidare la ricerca.

Definizione dei B⁺-alberi: condizioni - 1

- La struttura dei nodi interni (di ordine p) di un B⁺-albero è la seguente:

(1) ogni nodo interno del B⁺-albero ha la forma:

$$\langle P_1, K_1, P_2, K_2, \dots, P_{q-1}, K_{q-1}, P_q \rangle,$$

dove $q \leq p$ e ogni P_i è un *tree pointer* (puntatore ad un altro nodo del B⁺-albero)

(2) per ogni nodo interno, si ha che $K_1 < K_2 < \dots < K_{q-1}$

(3) ogni nodo interno ha al più p *tree pointer*

(4) per tutti i valori X della *search key* nel sottoalbero puntato da P_i , si ha che

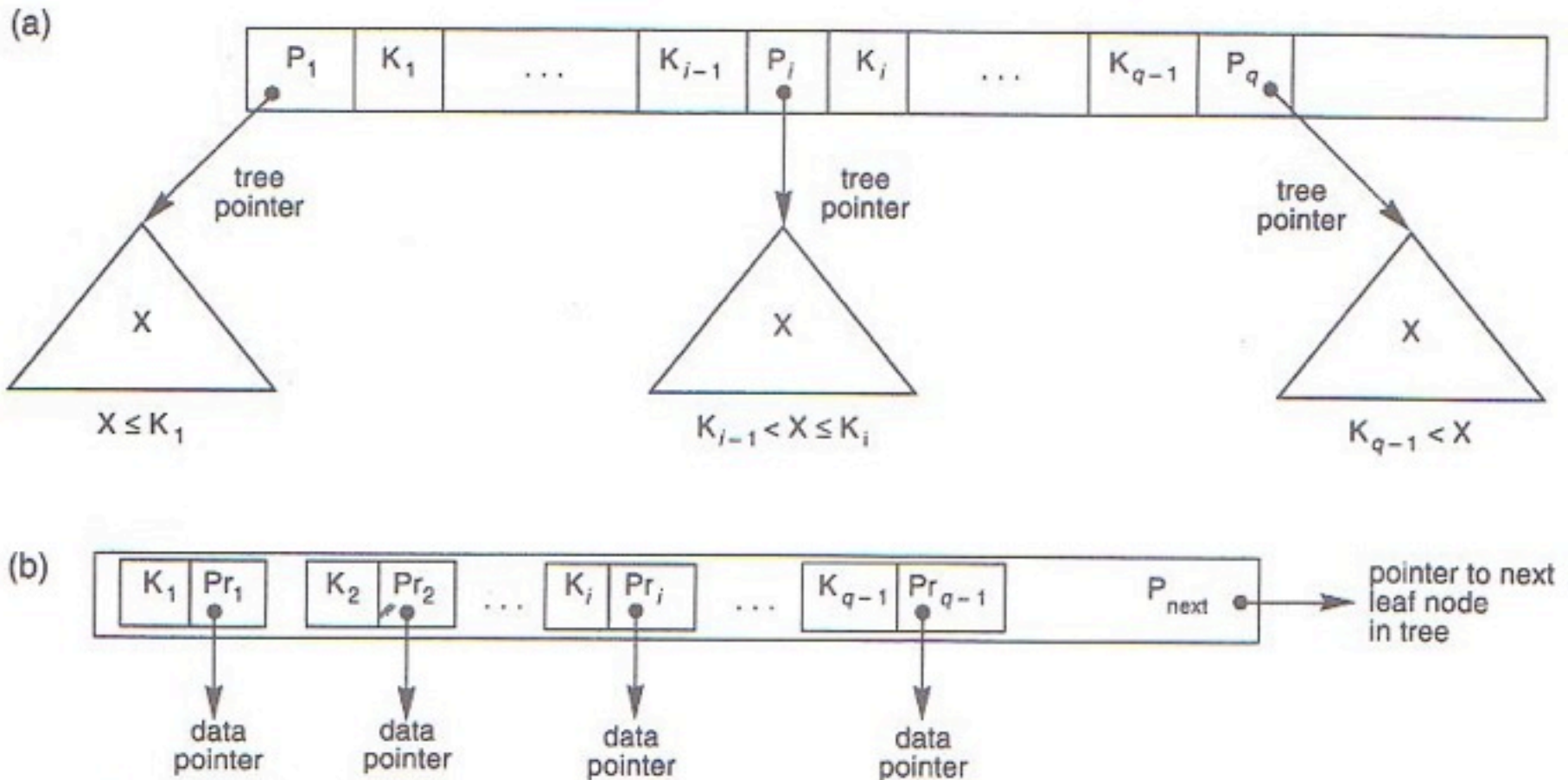
$$- K_{i-1} < X \leq K_i \quad \text{for } 1 < i < q,$$

$$- X \leq K_i \quad \text{for } i = 1,$$

$$- K_{i-1} < X \quad \text{for } i = q$$

B⁺-alberi: condizioni - 2

- (5) ogni nodo interno, esclusa la radice, ha almeno $\lceil p/2 \rceil$ *tree pointer*. La radice ha almeno 2 *tree pointer* se è un nodo interno.
- (6) un nodo interno con q *tree pointer*, con $q \leq p$, ha $q-1$ campi di ricerca.



B⁺-alberi: condizioni - 3

- La struttura dei nodi foglia (di ordine p_{leaf}) di un B⁺-albero è la seguente:

(1) ogni nodo foglia è della forma:

$$\langle \langle K_1, Pr_1 \rangle, \langle K_2, Pr_2 \rangle, \dots, \langle K_q, Pr_q \rangle P_{next} \rangle,$$

dove $q \leq p_{leaf}$, Pr_i è un *data pointer* e P_{next} è un *tree pointer* punta al successivo nodo foglia del B⁺-albero

(2) per ogni nodo, si ha che $K_1 < K_2 < \dots < K_q$

(3) ogni Pr_i è un *data pointer* che punta al record con valore del campo di ricerca uguale a K_i o ad un blocco contenente tale record (o ad un blocco di puntatori ai record con valore del campo di ricerca uguale a K_i , se il campo di ricerca non è una chiave)

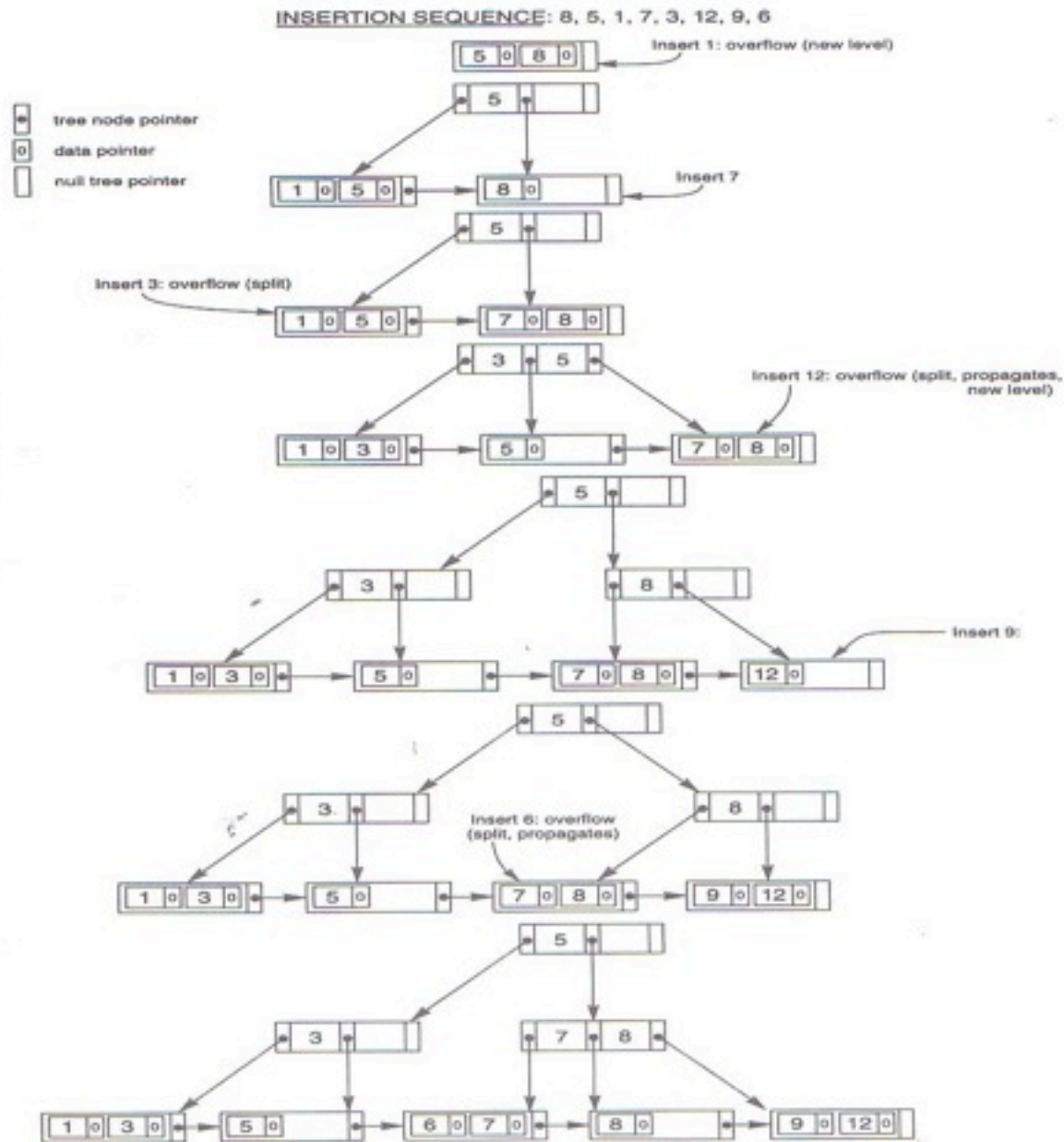
(4) ogni nodo foglia ha almeno $\lceil p_{leaf}/2 \rceil$ valori

(5) tutti i nodi foglia sono dello stesso livello

Generazione di un B⁺-alberi

- Per un B⁺-albero costruito su un campo chiave,
 - i puntatori appartenenti ai nodi interni sono *tree pointer* a blocchi che sono nodi dell'albero
 - i puntatori appartenenti ai nodi foglia sono *data pointer* a record (o blocchi) del file dei dati, ad eccezione di P_{next} che è un *tree pointer* al successivo nodo foglia

Esempio inserimento ($p=3$ e $p_{\text{leaf}} = 2$)



Esempio cancellazione ($p=3$ e $p_{\text{leaf}} = 2$)

